

Nios II Benchmarking - Cyclic Redundancy Check

Today's Agenda

Intended topics for today's session

- Basic Performance Evaluation
- Nios II Performance Optimization

FPGA-based SoC Design

International Master of Science in Electrical Engineering

Prof. Dr. C. Jakob

University of Applied Sciences Darmstadt

h_da

Faculty of Electrical Engineering and Information Technology

fbeit

Nios II Benchmarking

International Master of Science in Electrical Engineering

Prof. Dr. C. Jakob

University of Applied Sciences Darmstadt

h_da

Faculty of Electrical Engineering and Information Technology

fbeit

Nios II Performance Optimization

International Master of Science in Electrical Engineering

Prof. Dr. C. Jakob

University of Applied Sciences Darmstadt

h_da

Faculty of Electrical Engineering and Information Technology

fbeit

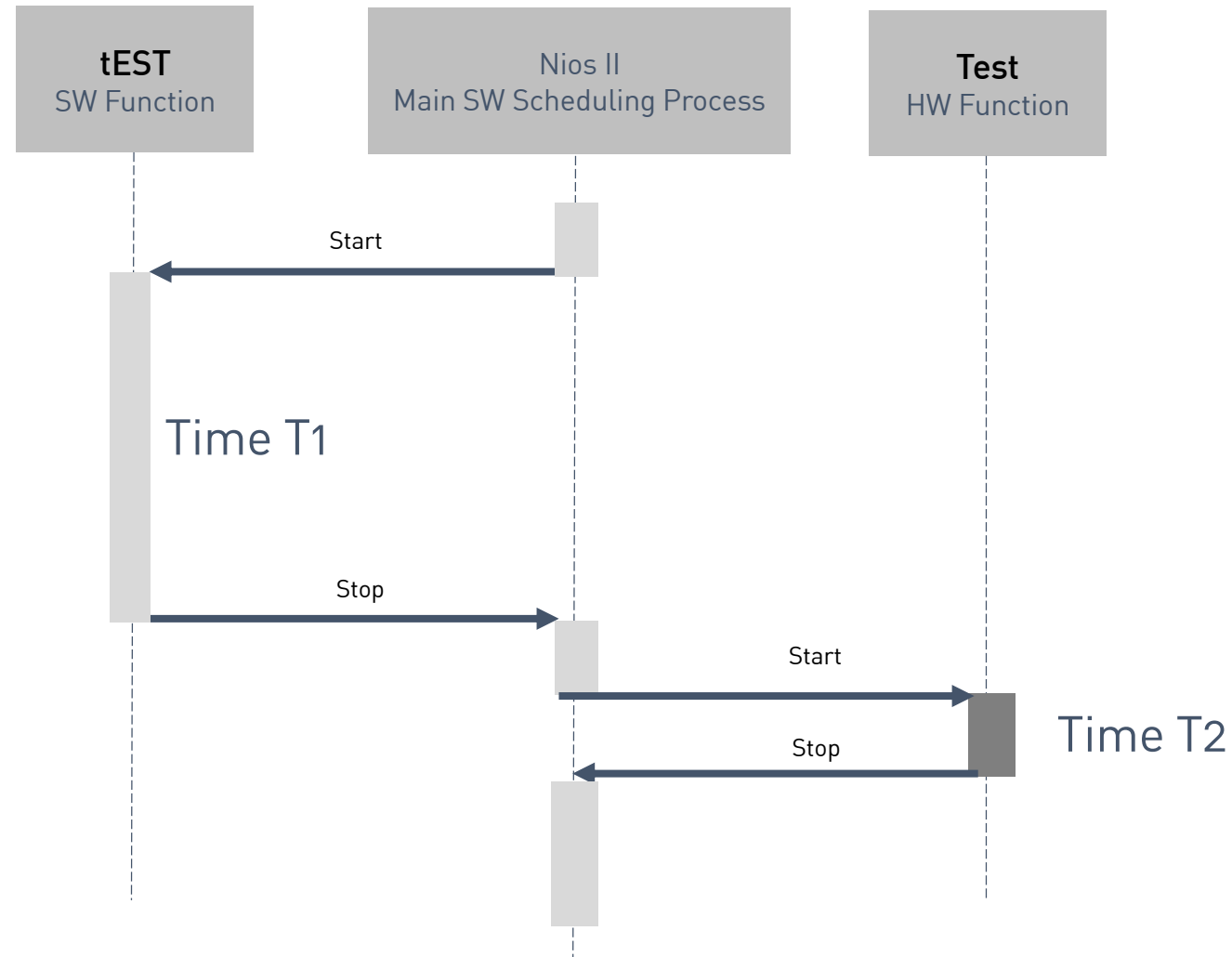
Basic Performance Evaluation

Notes

- Determine the differences in performance by measuring the respective execution times ...

Boost Factor: $T1/T2$

- We are evaluating different hardware/software implementations for the CRC computation ...



Nios II Performance Optimization

h_da – fbeit - FPGA-based SoC Design

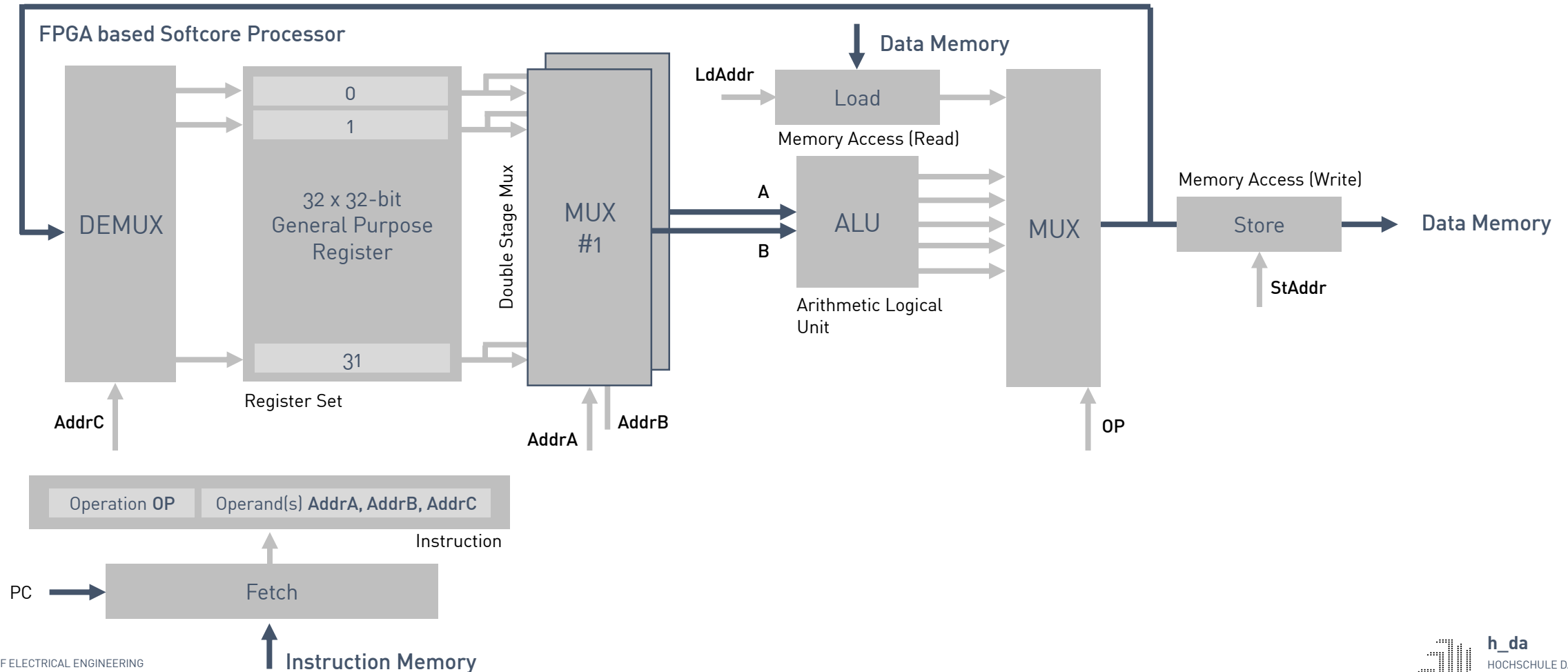
Nios II – Custom Instruction Set Extensions

Nios II Benchmarking - Cyclic Redundancy Check

Nios II Performance Optimization

h_da – fbeit - FPGA-based SoC Design

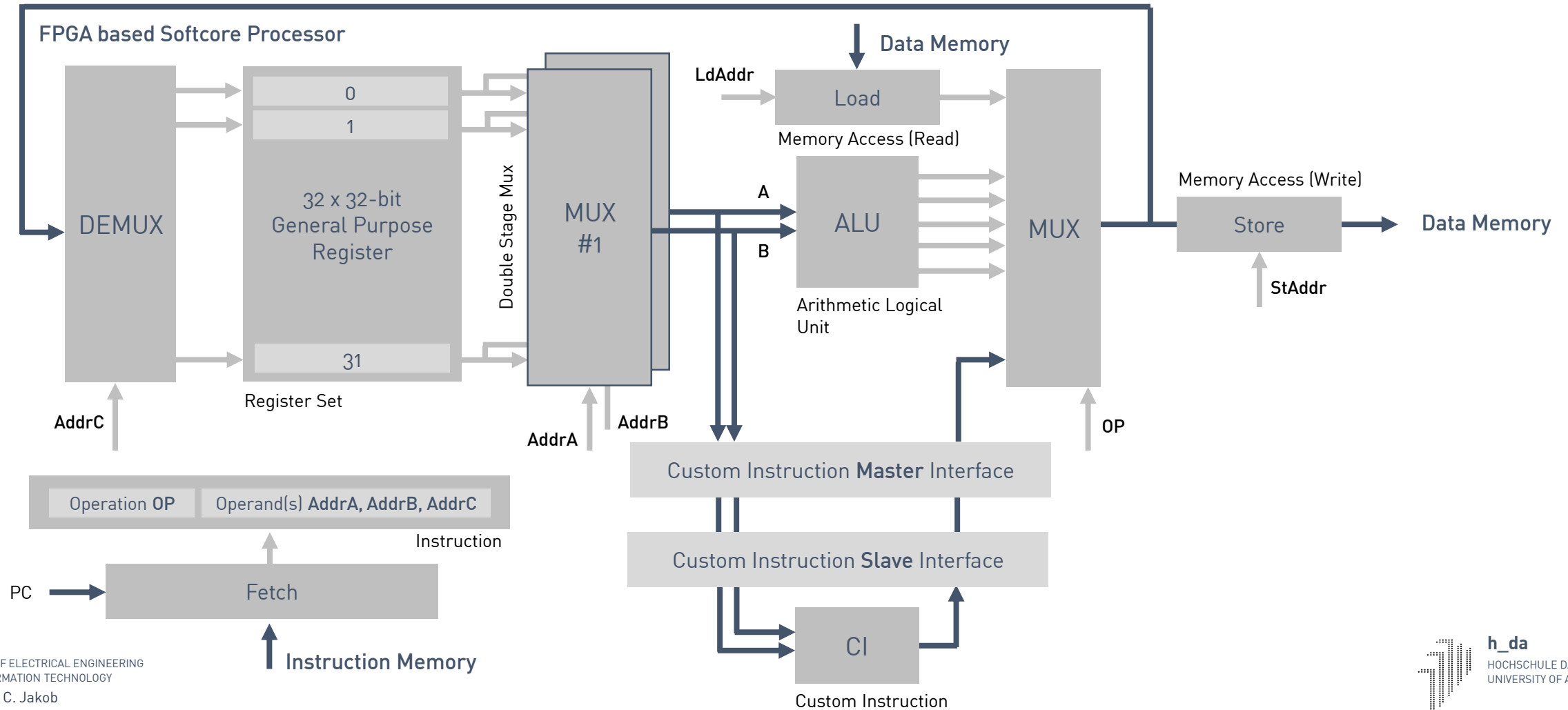
Nios II – Basic CPU Dataflow Diagram



Nios II Performance Optimization

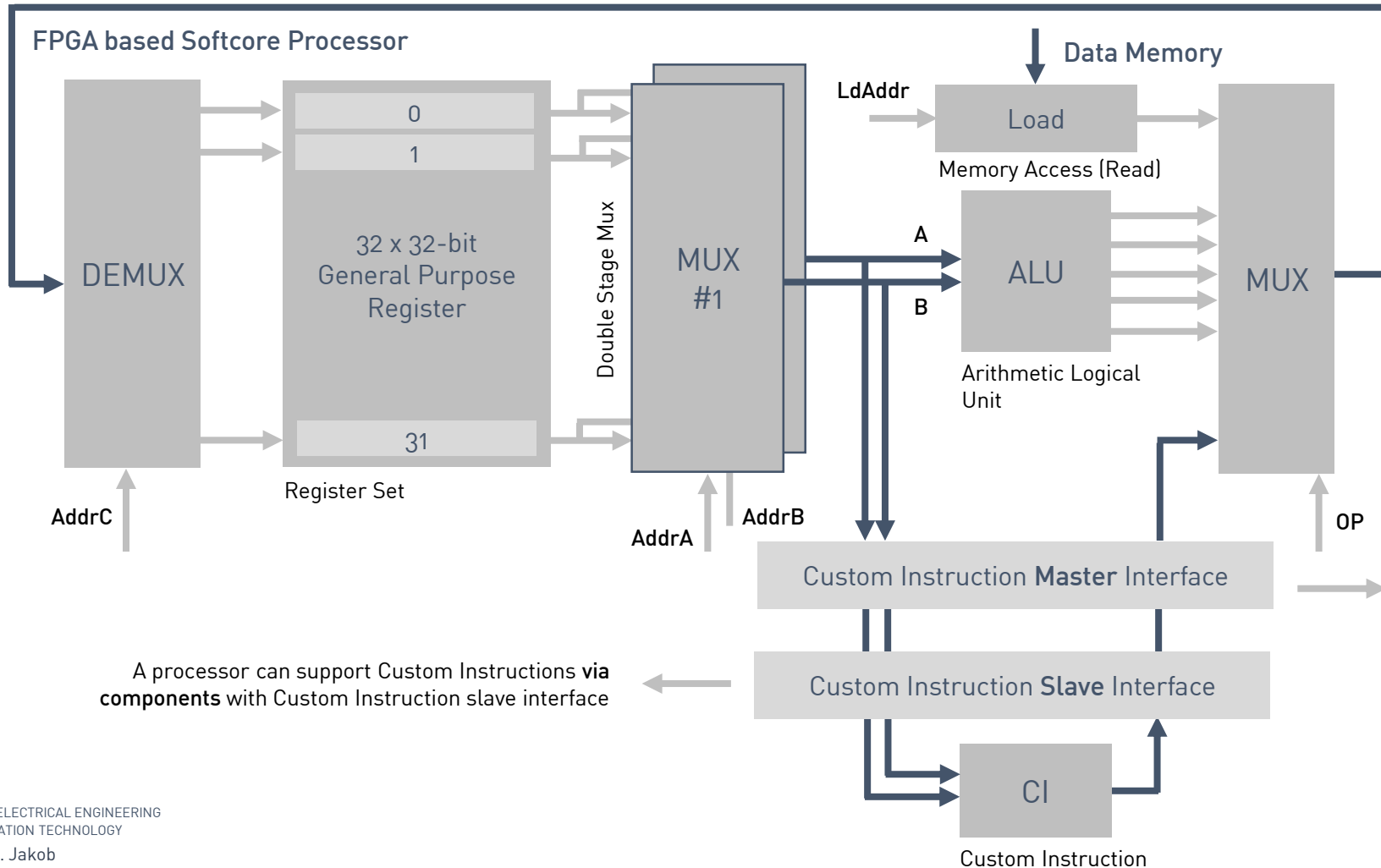
h_da – fbeit - FPGA-based SoC Design

Nios II – Custom Instruction Set Extensions



Nios II – Custom Instruction Set Extensions

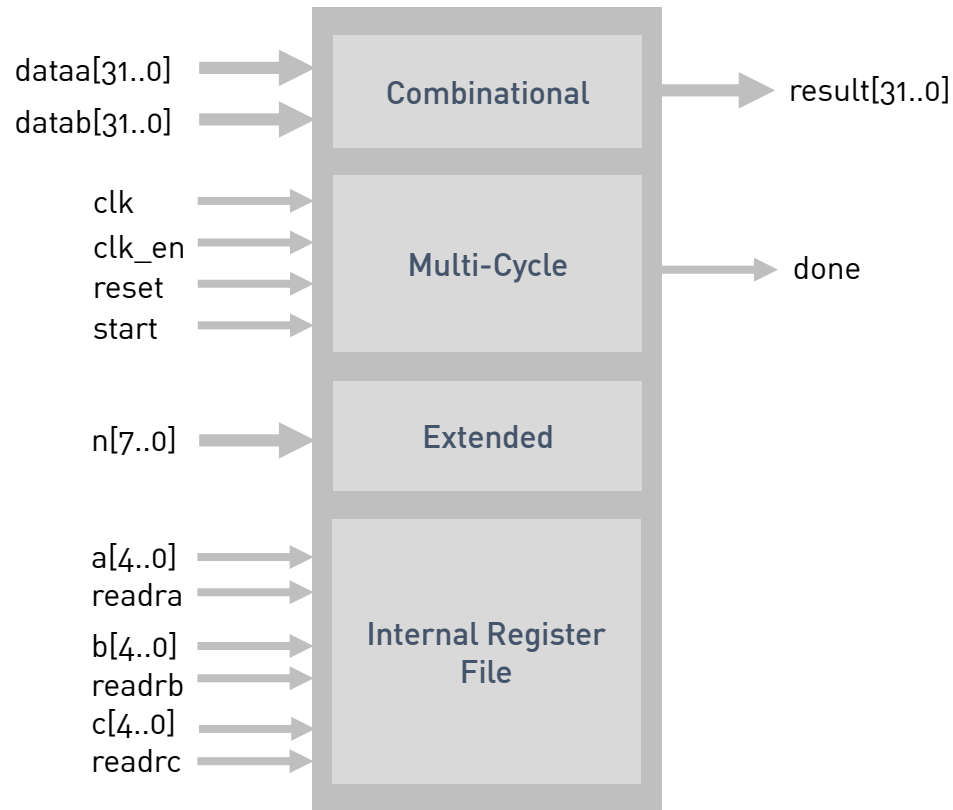
the memory interfaces have been removed in the figure for the sake of clarity



Nios II Custom Instruction Set Extension

- Each custom instruction contains at least one custom instruction slave port, through which it connects to the ALU.
- A custom instruction slave is another slave interface that receives the data input signals and returns the result.
- The custom instruction master is the master port of the processor that connects to the custom instruction.

Nios II – Custom Instruction Set Extensions



Combinational Custom Instruction

- A combinational custom instruction is a logic block that completes its logic function in a single clock cycle

Multicycle (Variable or fixed) Custom Instruction

- Multicycle (sequential) custom instructions consist of a logic block that requires two or more clock cycles to complete an operation

Extended (Multicycle or Combinational) Custom Instruction

- An extended custom instruction allows a single custom logic block to implement several different operations

Internal Register File Custom Instruction

- The Nios II Gen2 processor allows custom instruction logic to access its own internal register file

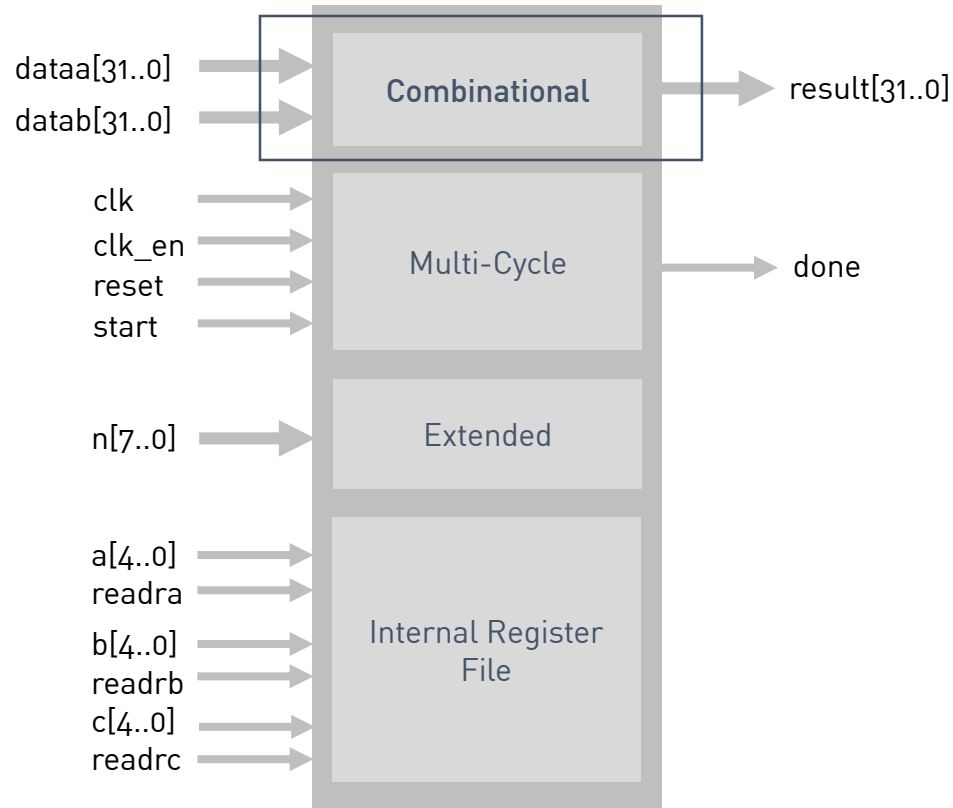
External Interface Custom Instruction

- Nios II external interface custom instructions allow you to add an interface to communicate with logic outside of the processor's datapath.

Nios II Performance Optimization

h_da – fbeit - FPGA-based SoC Design

Combinational Custom Instruction



Combinational Custom Instruction

- A combinational custom instruction is a logic block that completes its logic function in a single clock cycle

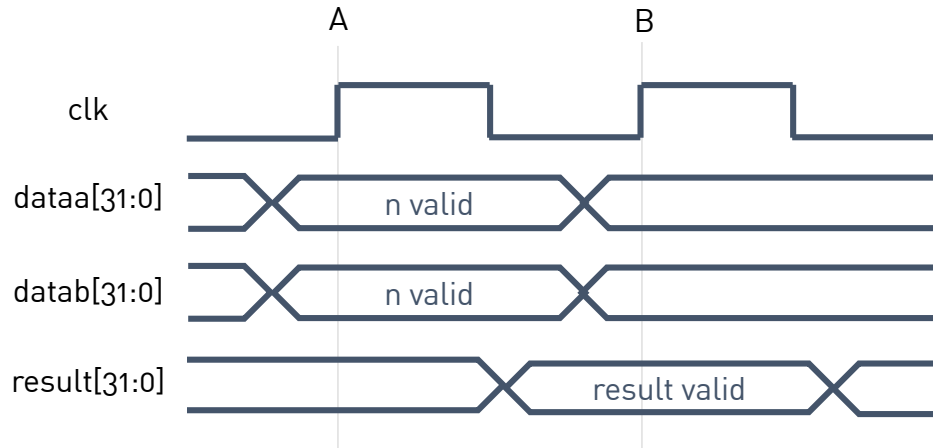
Combinational Custom Instruction

- `dataa[31:0]` Input operand to custom instruction
- `datab[31:0]` Input operand to custom instruction
- `result[31:0]` Result of custom instruction

Nios II Performance Optimization

h_da – fbeit - FPGA-based SoC Design

Combinational Custom Instruction

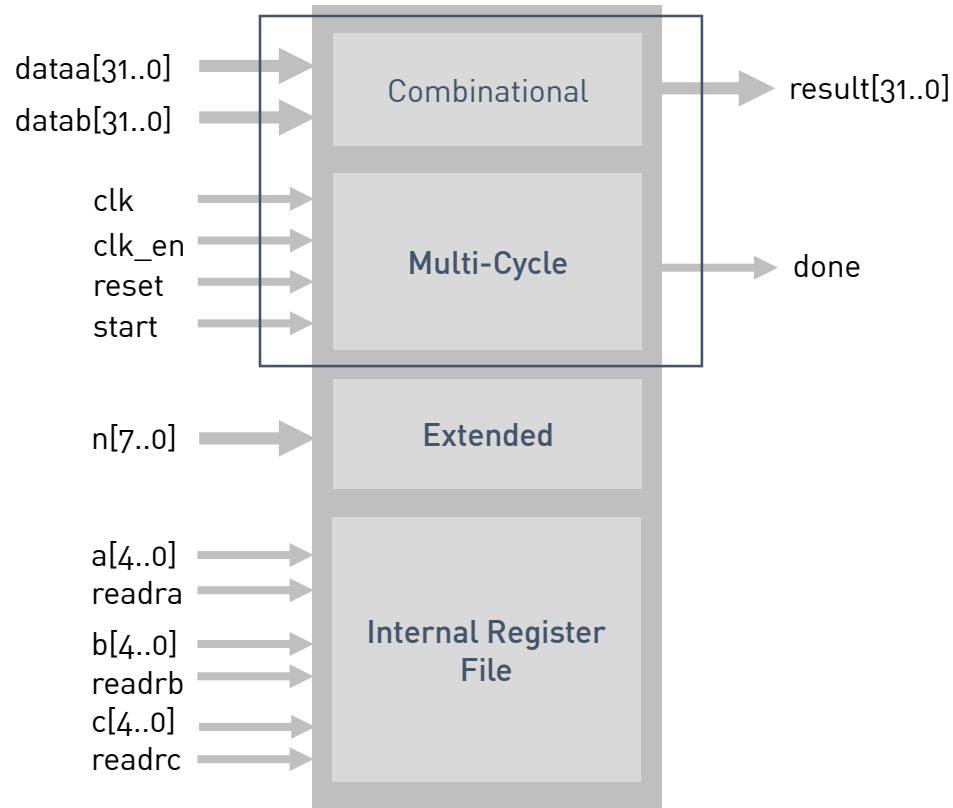


Timing Diagram: Nios II Custom Instruction

Combinational Custom Instruction

- A combinational custom instruction is a logic block that completes its logic function in a single clock cycle
- The processor presents the input data on the dataa and datab ports on the rising edge of the processor clock (A).
- The processor reads the result port on the rising edge of the following processor clock cycle (B).

Multicycle Custom Instruction



Multicycle (Variable or fixed) Custom Instruction

- Multicycle (sequential) custom instructions consist of a logic block that requires two or more clock cycles to complete an operation

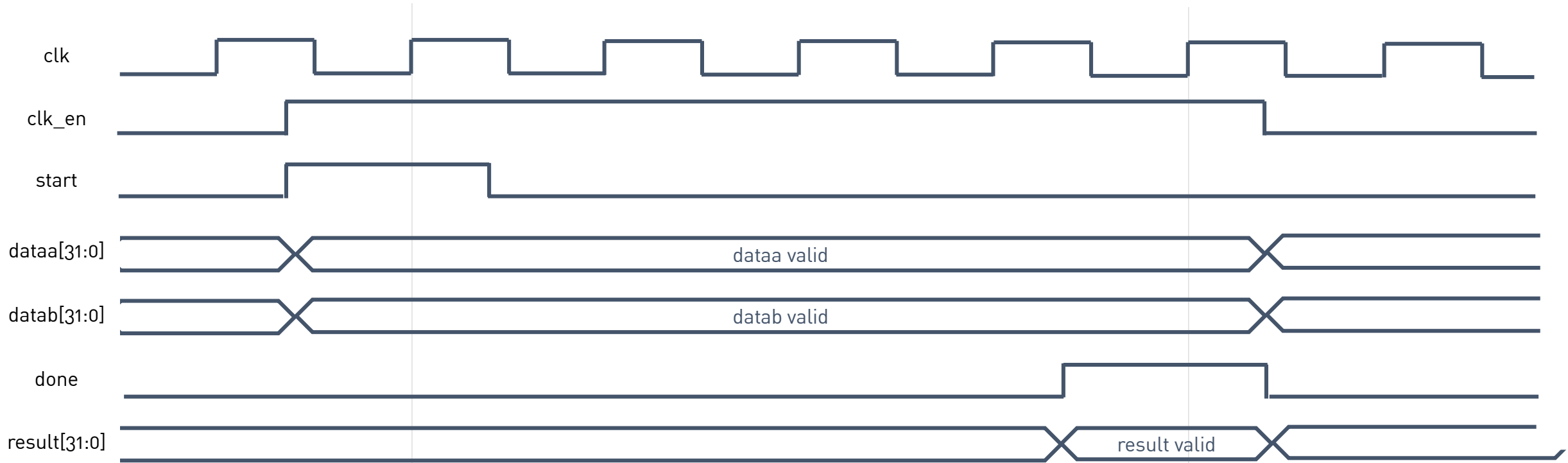
Multicycle (Variable or fixed) Custom Instruction Ports

- | | |
|-----------------------------|--|
| ▪ <code>clk</code> | System clock |
| ▪ <code>clk_en</code> | Clock enable |
| ▪ <code>reset</code> | Synchronous reset |
| ▪ <code>start</code> | Commands CI logic to start execution |
| ▪ <code>done</code> | CI logic indicates to the CPU that execution is complete |
| ▪ <code>dataa[31:0]</code> | Input operand to custom instruction |
| ▪ <code>datab[31:0]</code> | Input operand to custom instruction |
| ▪ <code>result[31:0]</code> | Result of custom instruction |

Nios II Performance Optimization

h_da – fbeit - FPGA-based SoC Design

Multicycle Custom Instruction

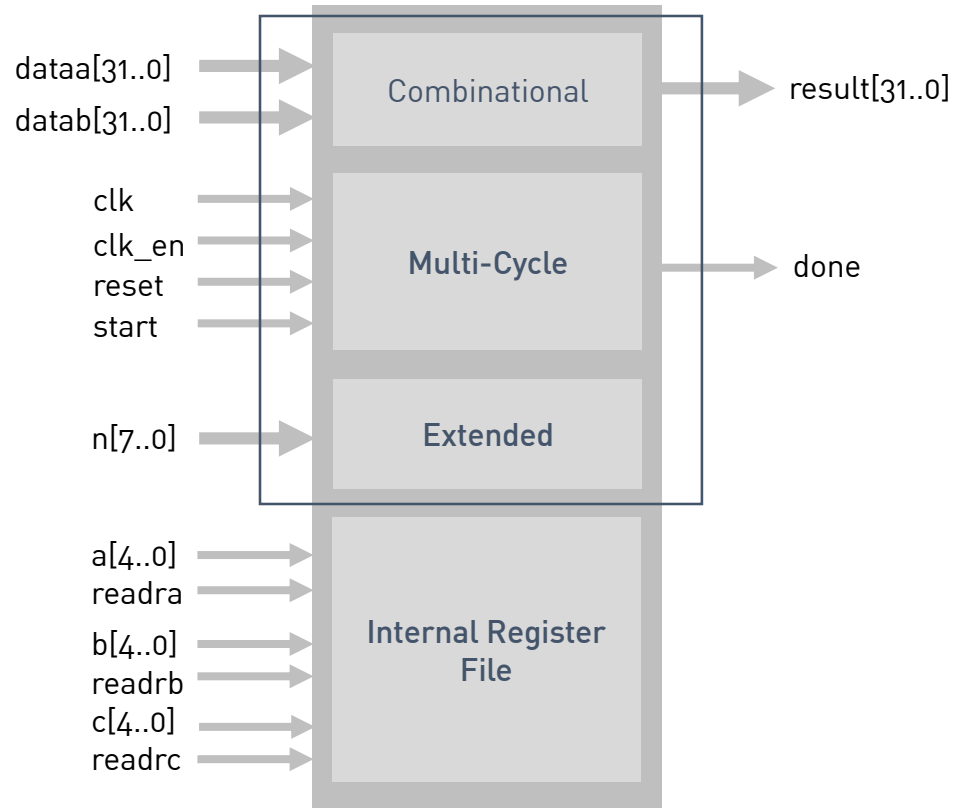


Timing Diagram: Nios II Custom Instruction



Fixed Duration of four clock cycles

Extended Custom Instruction



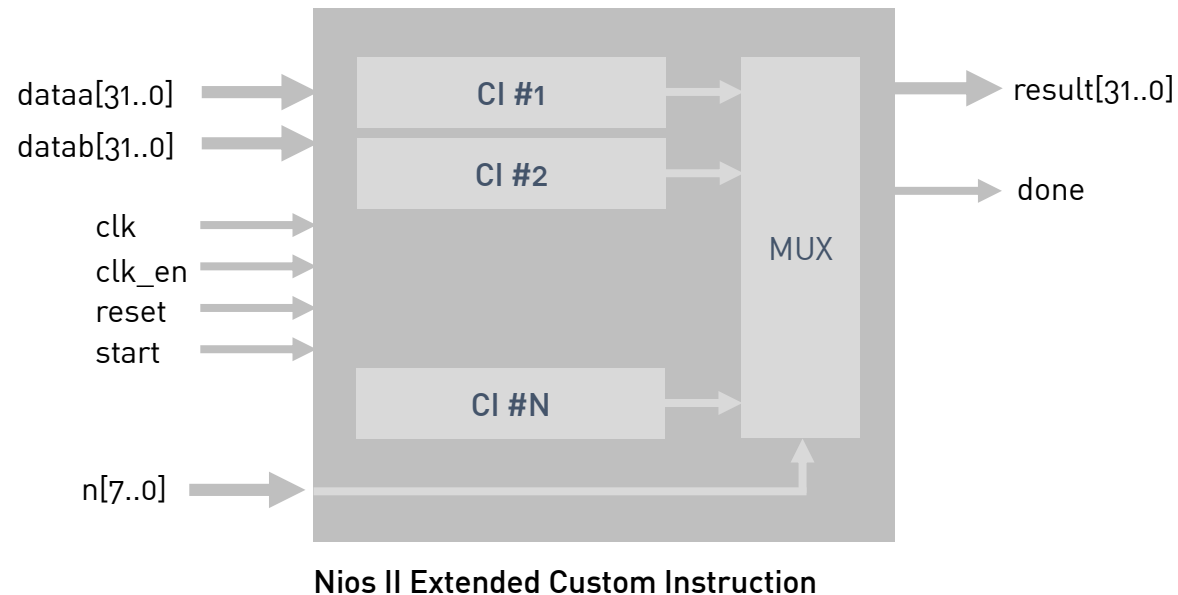
Extended (Multicycle) Custom Instruction

- An extended custom instruction allows a single custom logic block to implement several different operations
- `N` is asserted together with `start`, and remains stable for the entire duration of the custom instruction

Extended (Multicycle) Custom Instruction Ports

- `clk` System clock
- `clk_en` Clock enable
- `reset` Synchronous reset
- `start` Commands CI logic to start execution
- `done` CI logic indicates to the CPU that execution is complete
- `n[7:0]` CI selection index
- `dataa[31:0]` Input operand to custom instruction
- `datab[31:0]` Input operand to custom instruction
- `result[31:0]` Result of custom instruction

Extended Custom Instruction



Extended (Multicycle) Custom Instruction

- An extended custom instruction allows a single custom logic block to implement several different operations
- N is asserted together with start, and remains stable for the entire duration of the custom instruction

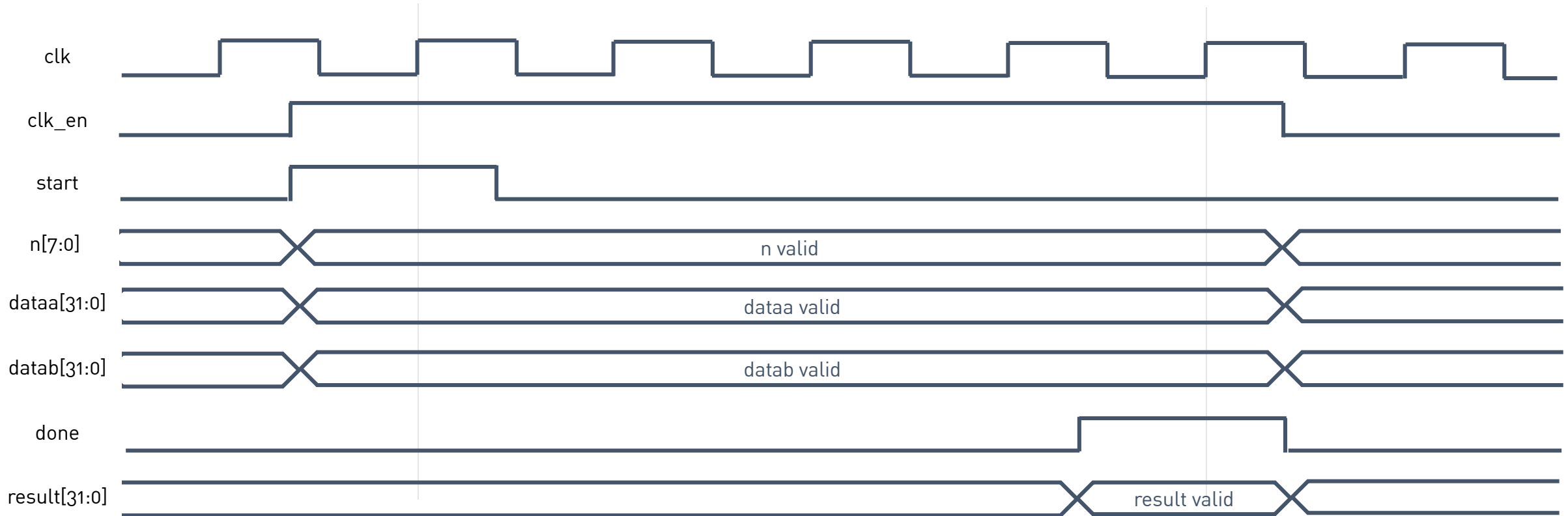
Extended (Multicycle) Custom Instruction Ports

- `clk` System clock
- `clk_en` Clock enable
- `reset` Synchronous reset
- `start` Commands CI logic to start execution
- `done` CI logic indicates to the CPU that execution is complete
- `n[7:0]` CI selection index
- `dataa[31:0]` Input operand to custom instruction
- `datab[31:0]` Input operand to custom instruction
- `result[31:0]` Result of custom instruction

Nios II Performance Optimization

h_da – fbeit - FPGA-based SoC Design

Extended Custom Instruction

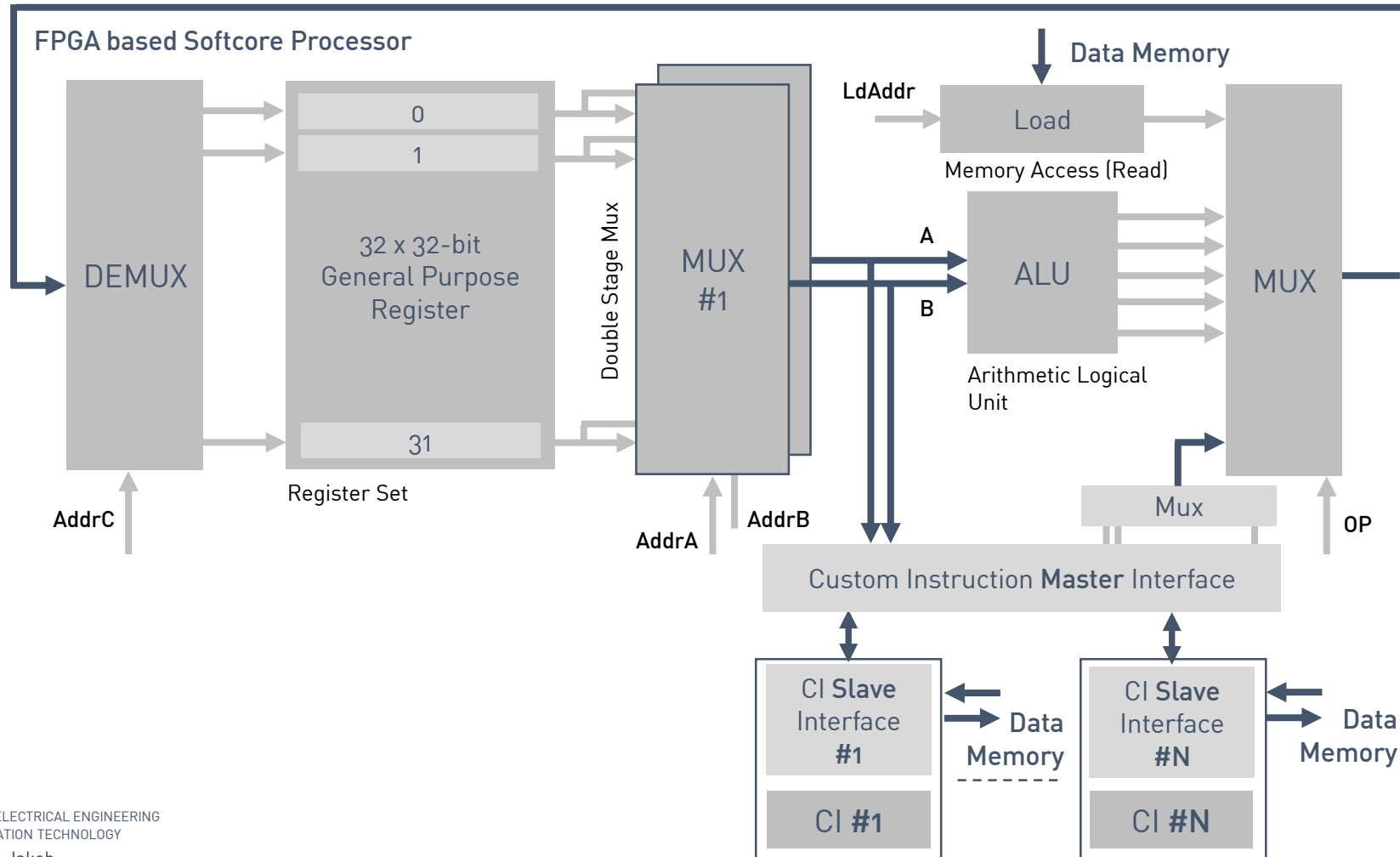


Timing Diagram: Nios II Custom Instruction

Nios II Performance Optimization

h_da – fbeit - FPGA-based SoC Design

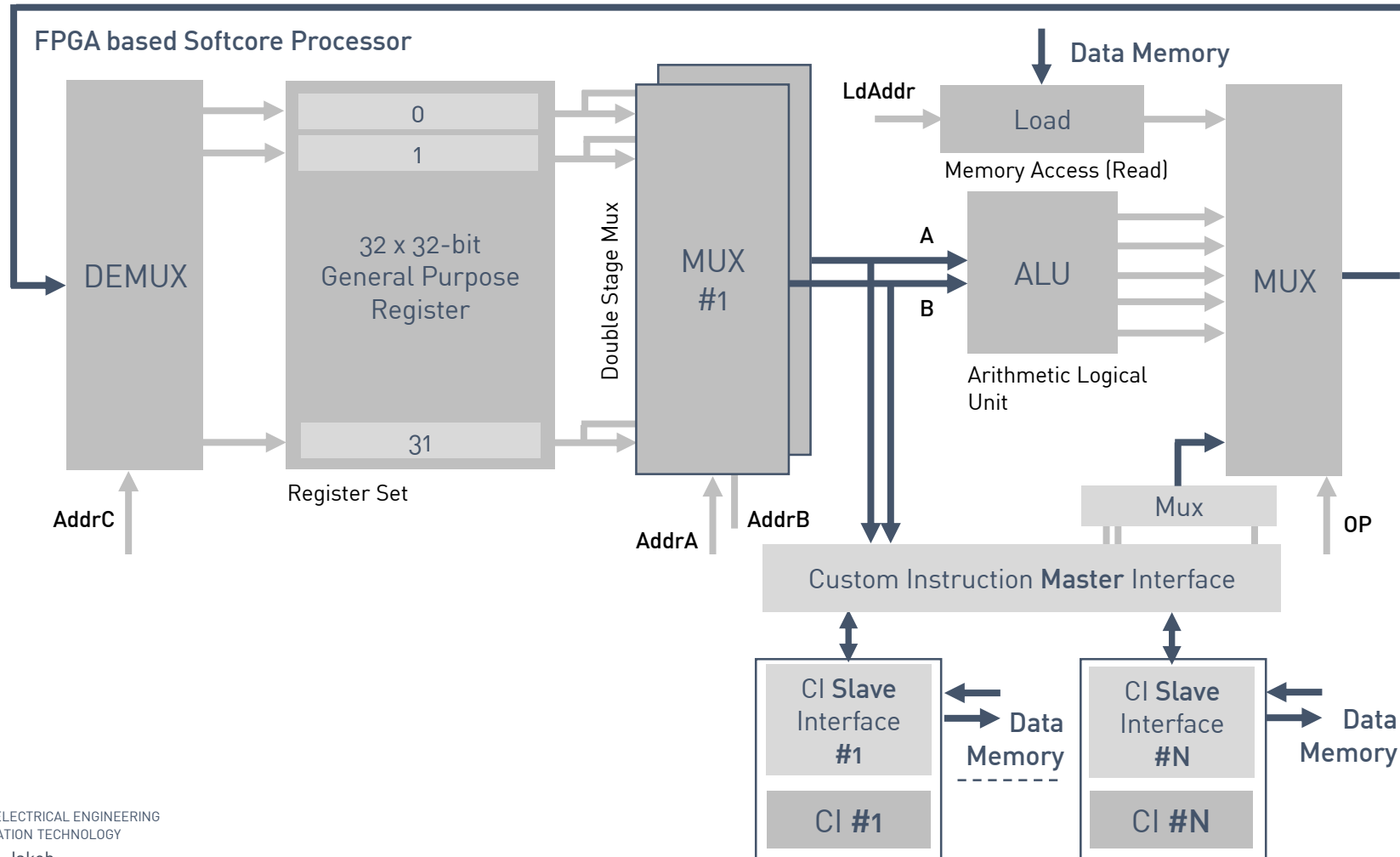
Your System might implement up to 256 different custom instructions



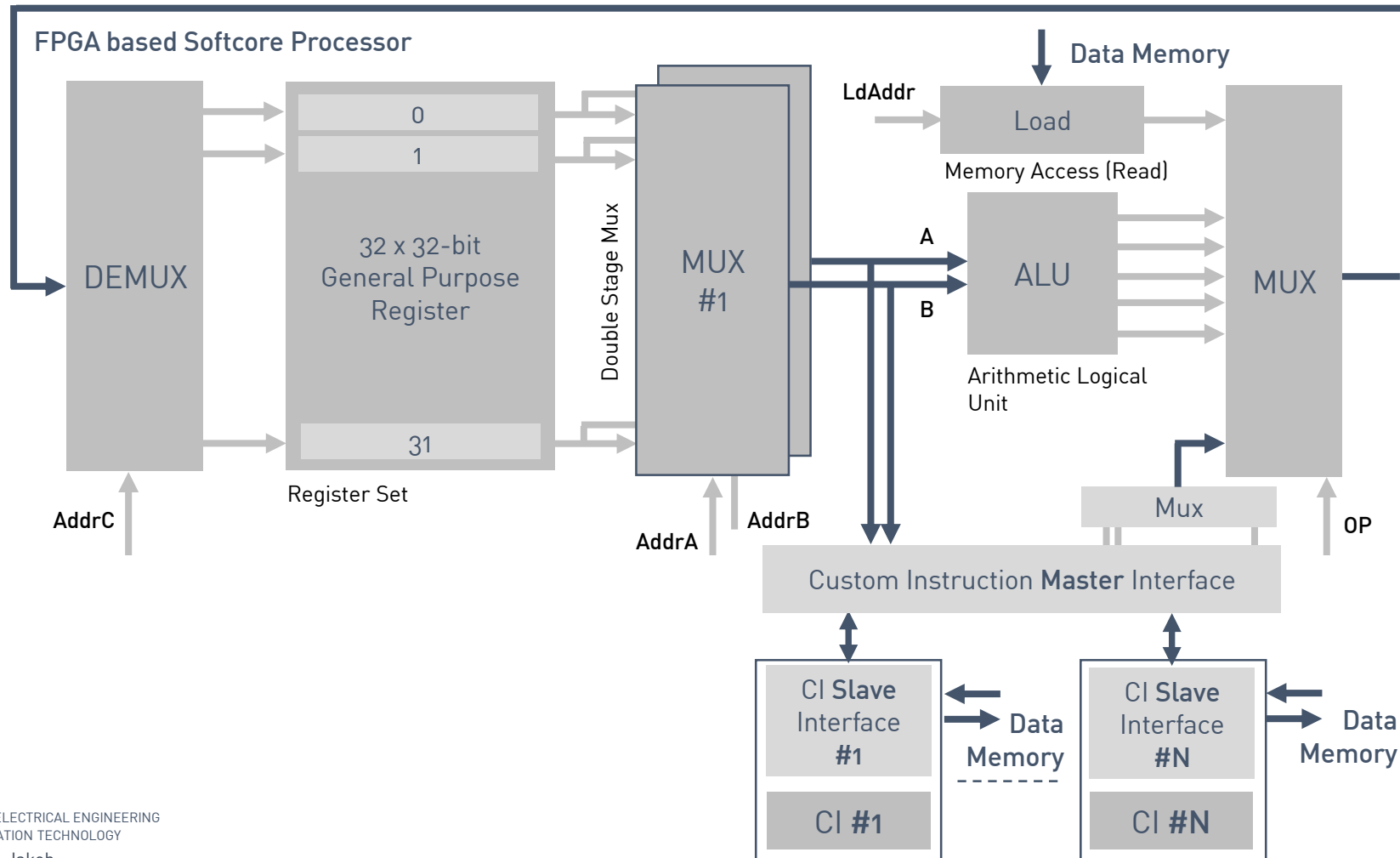
Nios II Performance Optimization

h_da – fbeit - FPGA-based SoC Design

Performance Limitations?



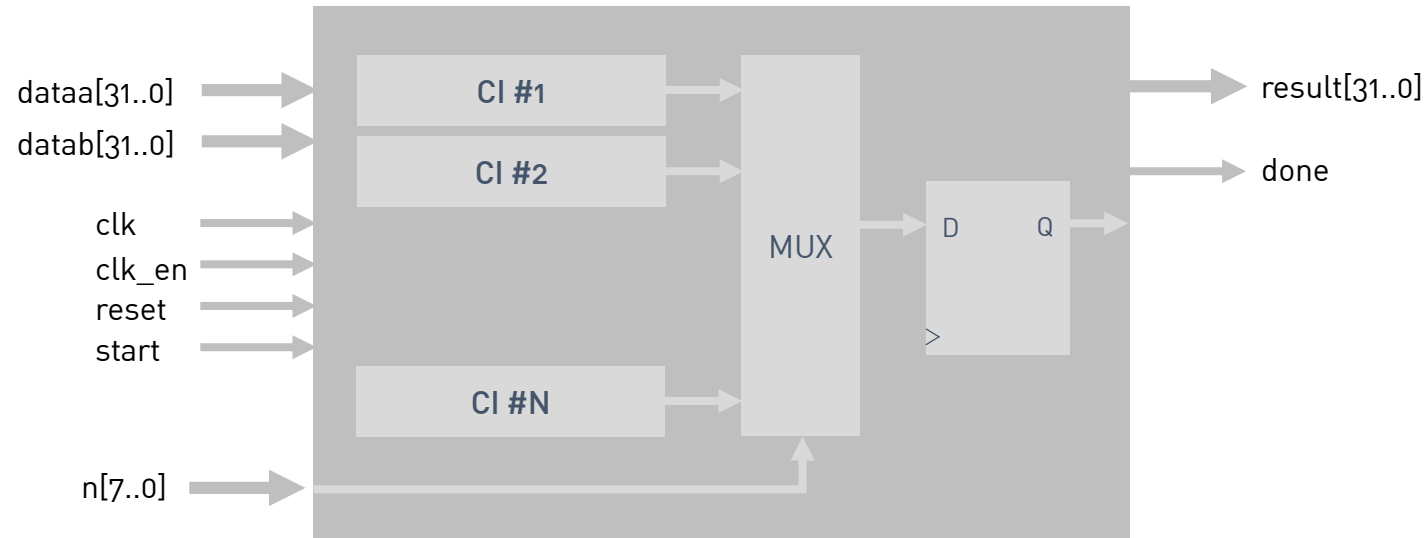
Performance Limitations?



CI/CPU Performance Issues

- Because custom instructions extend the Nios II processor's ALU, the logic must meet timing or the f_{MAX} of the processor will suffer.
- As you add custom instructions to the processor, the ALU multiplexer grows in width as the figure illustrates.
- Although you can pipeline custom instructions, you have no control over the automatically inserted ALU multiplexer.
- **As a result, you cannot pipeline the multiplexer for higher performance.**

Performance Limitations?



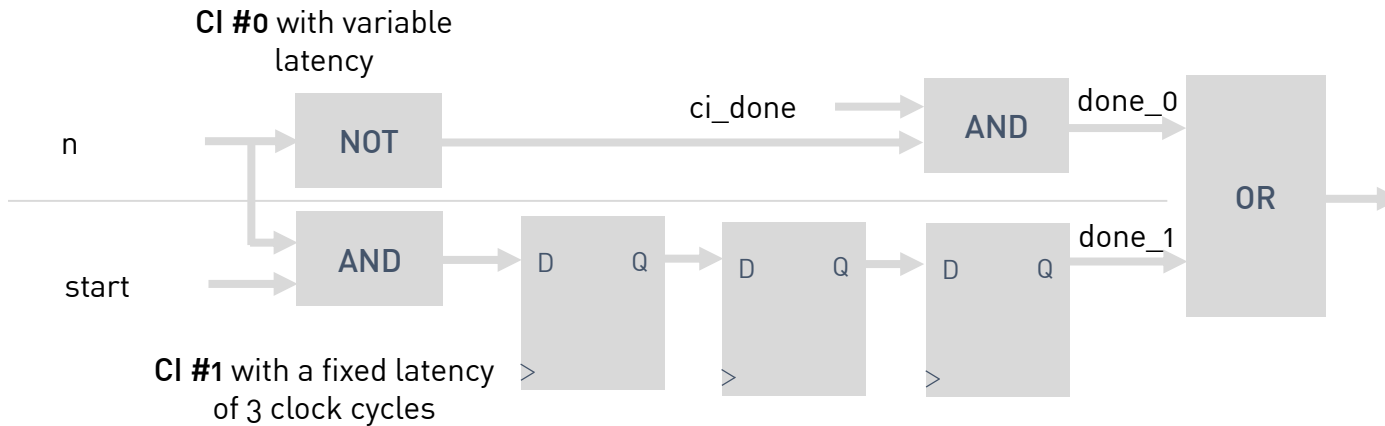
Nios II Extended Custom Instruction

- With multiple custom instructions built into a logic block, you can pipeline the output if it fails timing. To combine custom instructions, each must have identical latency characteristics.

CI/CPU Performance Issues

- Instead of adding several custom instructions, you can combine the functionality into a single logic block as shown on the right hand side. When you combine custom instructions you use selector bits to select the required functionality.
- If you create a combined custom instruction, you must insert the multiplexer in your logic manually. This approach gives you full control over the multiplexer logic that generates the output.
- You can pipeline the multiplexer to prevent your combined custom instruction from becoming part of a critical timing path.
- Greater Control over the delay added to the EX stage

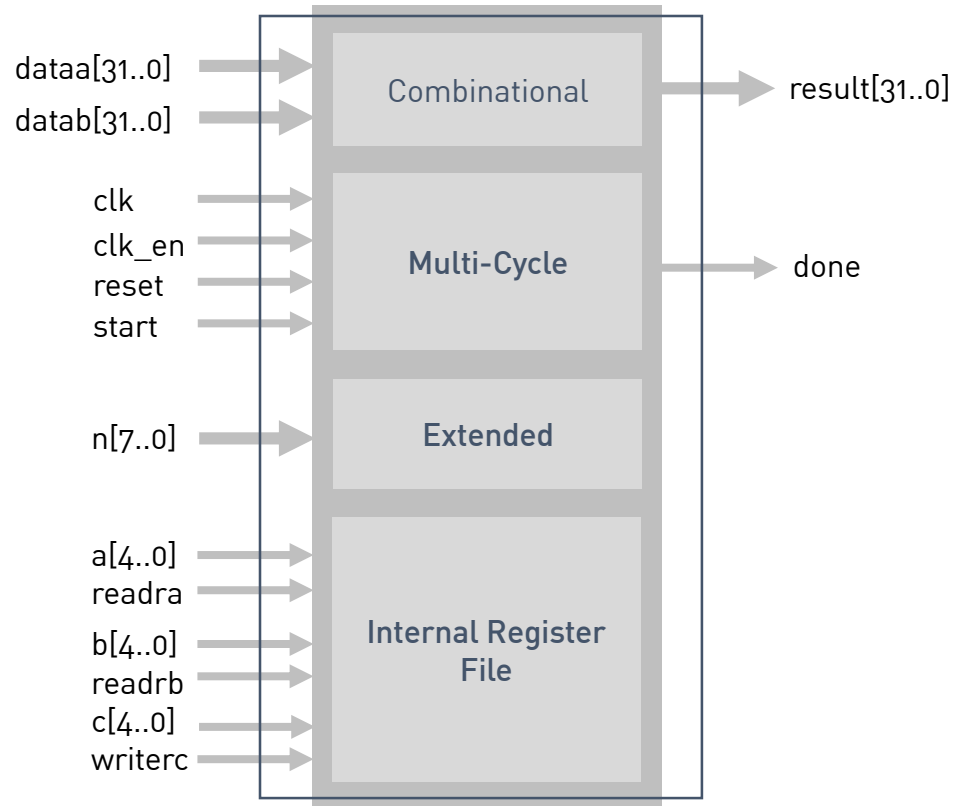
Combining CIs with different Latencies



CI/CPU Performance Issues

- Custom instructions are either fixed latency or variable latency.
- You can convert fixed latency custom instructions to variable latency by adding timing logic. The figure shows the simplest method to implement this conversion by shifting the start bit by $\langle n \rangle$ clock cycles and logically ORing all the done bits.

Custom Instruction with Internal Register Set



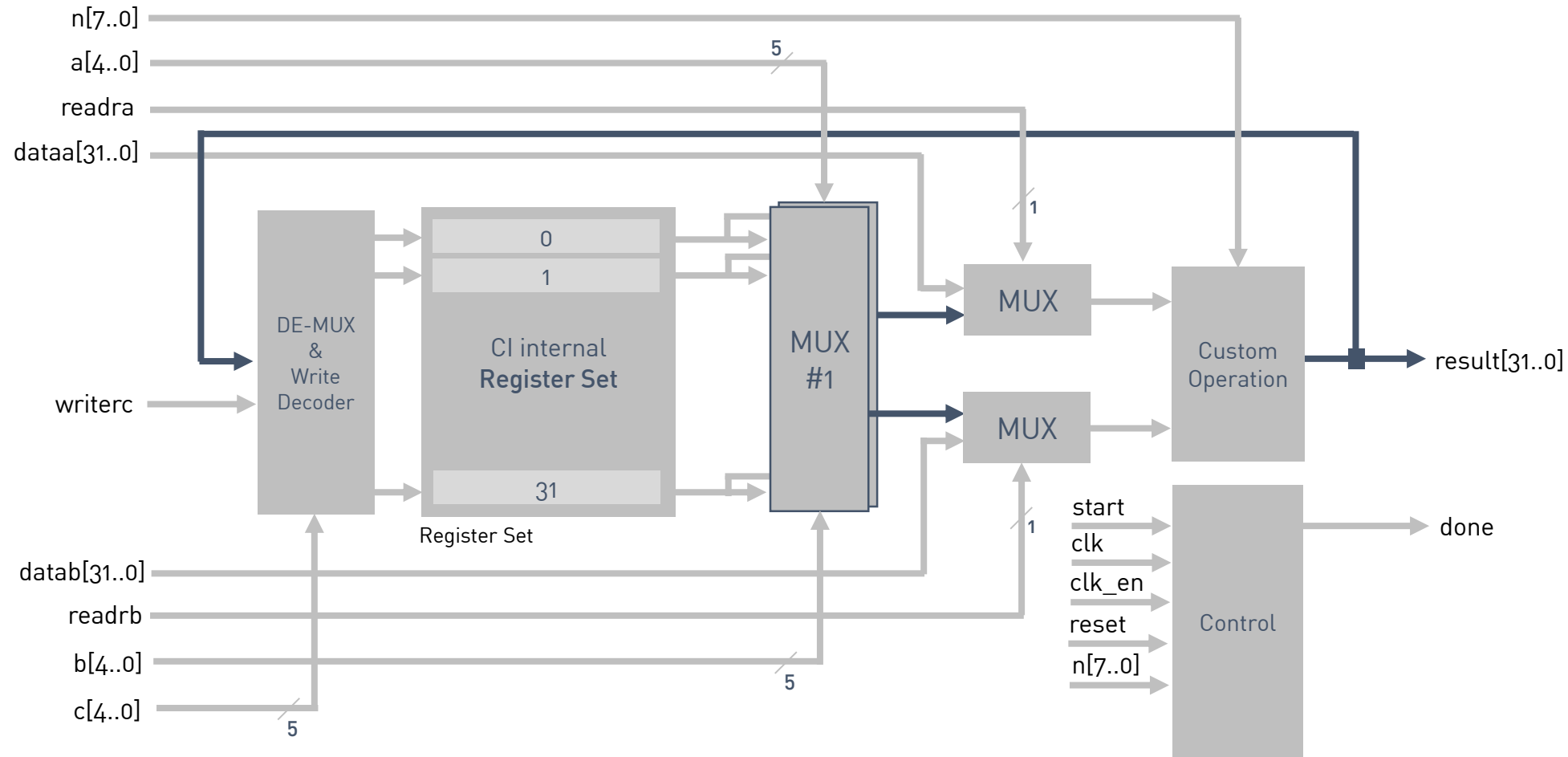
Internal Register File Custom Instruction Ports

- **clk** System clock
- **clk_en** Clock enable
- **reset** Synchronous reset
- **start** Commands CI logic to start execution
- **done** CI indicates to the CPU that execution is complete
- **n[7:0]** CI selection index
- **dataa[31:0]** Input operand to custom instruction
- **datab[31:0]** Input operand to custom instruction
- **result[31:0]** Result of custom instruction
- **a[4:0]** CI internal register number for data source A
- **b[4:0]** CI internal register number for data source B
- **c[4:0]** CI internal register number for the data destination
- **readra/b** If readra/b is high, the Nios II processor register a/b supplies dataa/datab. If reada/b is low, the CI logic reads the internal register a/b
- **writerc** If writerc is high, the Nios II processor writes the value on the result port to register c. If writerc is low, CI logic writes to internal register c

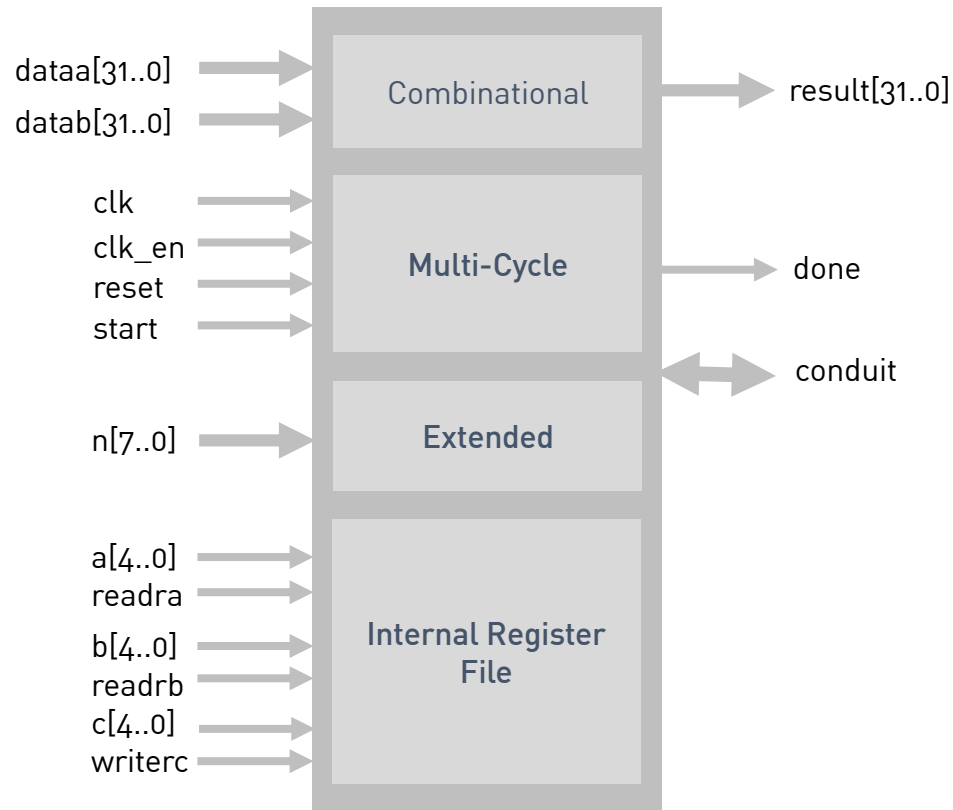
Nios II Performance Optimization

h_da – fbeit - FPGA-based SoC Design

Custom Instruction with Internal Register Set



Custom Instruction with External Interfaces



External Interface Custom Instructions

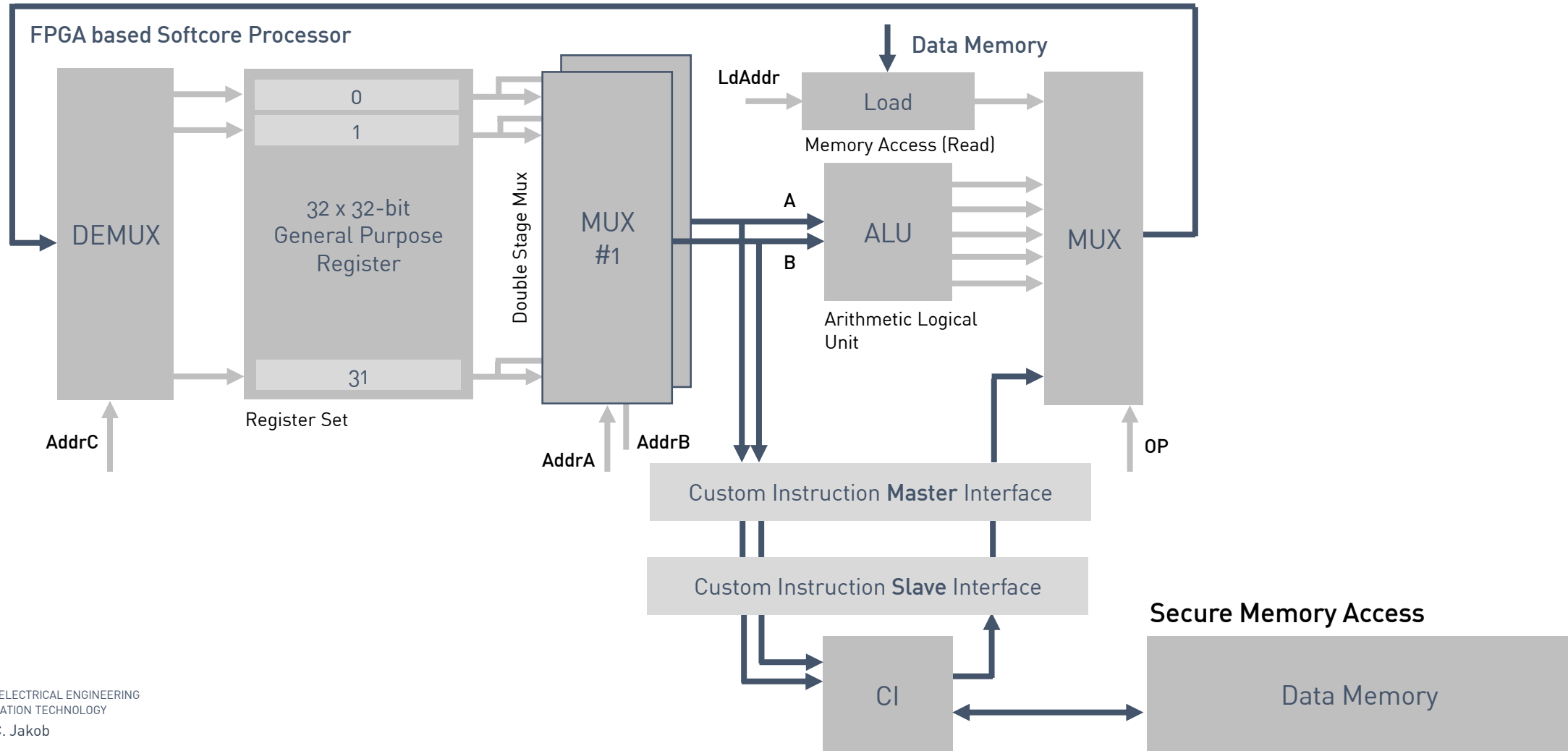
- Nios II **CI external interface** allow you to add an interface to communicate with logic outside of the processor's datapath.
- At system generation, conduits propagate out to the top level of the Qsys system, where external logic can access the signals.
- By enabling custom instruction logic to access memory external to the processor, external interface custom instructions extend the capabilities of the custom instruction logic.
- A combinational custom instruction must not have side effects. **In particular, a combinational custom instruction cannot have an external interface.** This restriction exists because the Nios II processor issues combinational custom instructions speculatively, to optimize execution. It issues the instruction before knowing whether it is necessary, and ignores the result if it is not required.

Nios II Performance Optimization

h_da – fbeit - FPGA-based SoC Design

Embedded Security Research Project - 2022

the memory interfaces have been removed in the figure
for the sake of clarity



Nios II Performance Optimization

h_da – fbeit - FPGA-based SoC Design

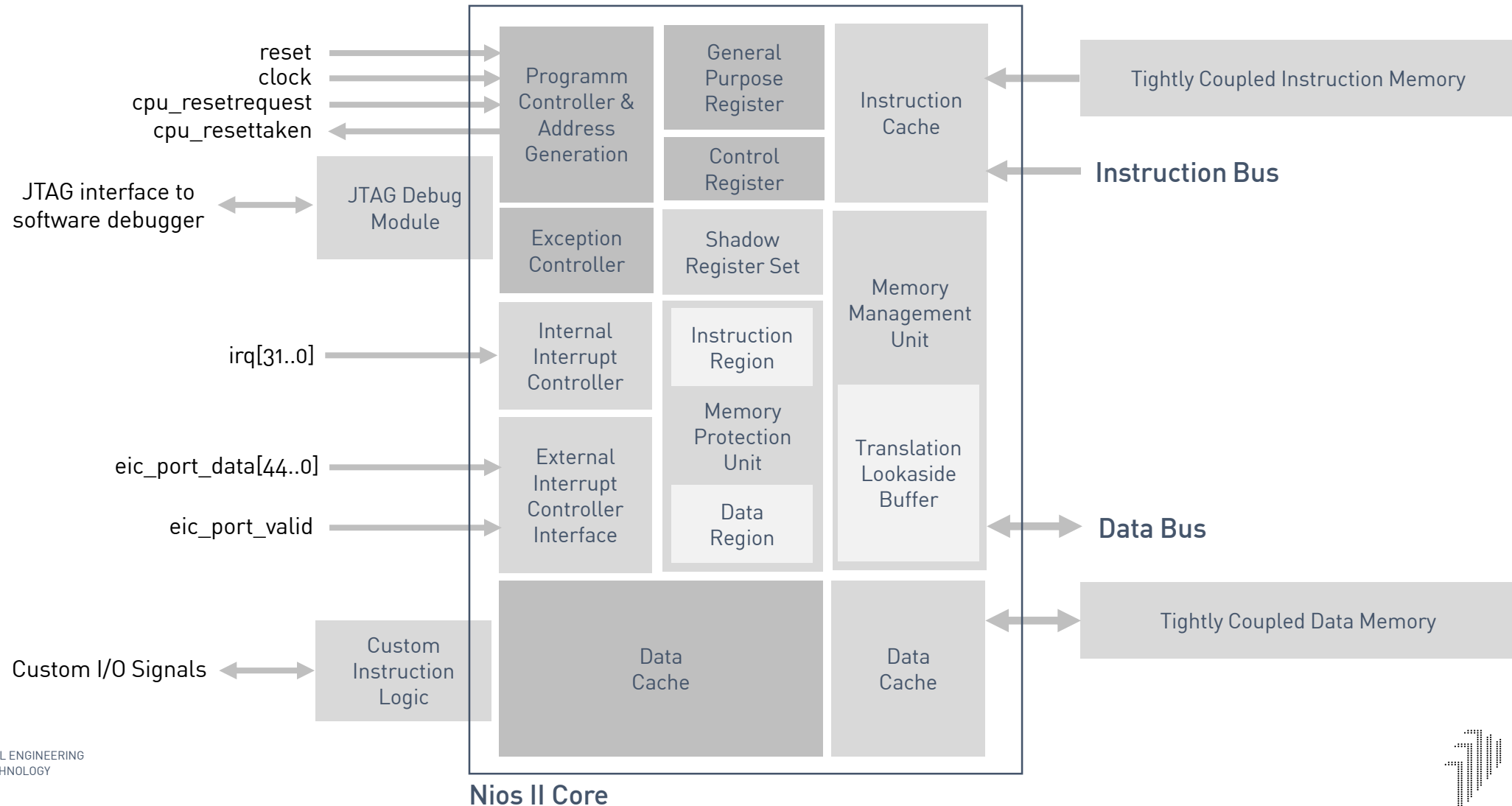
Further Performance Improvement Strategies

Nios II Benchmarking - Cyclic Redundancy Check

Nios II Performance Optimization

h_da – fbeit - FPGA-based SoC Design

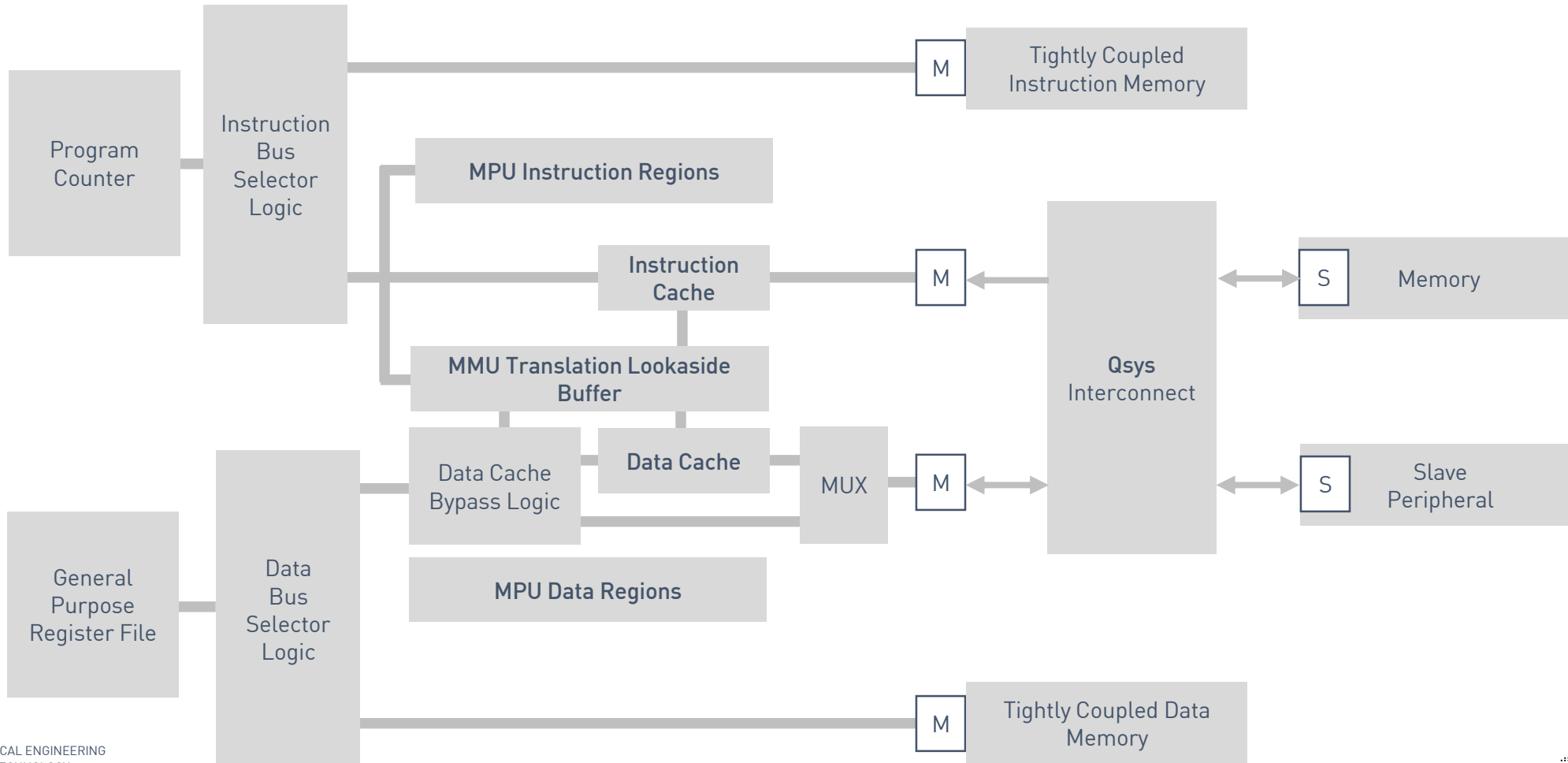
Nios II Core



Nios II Performance Optimization

h_da – fbeit - FPGA-based SoC Design

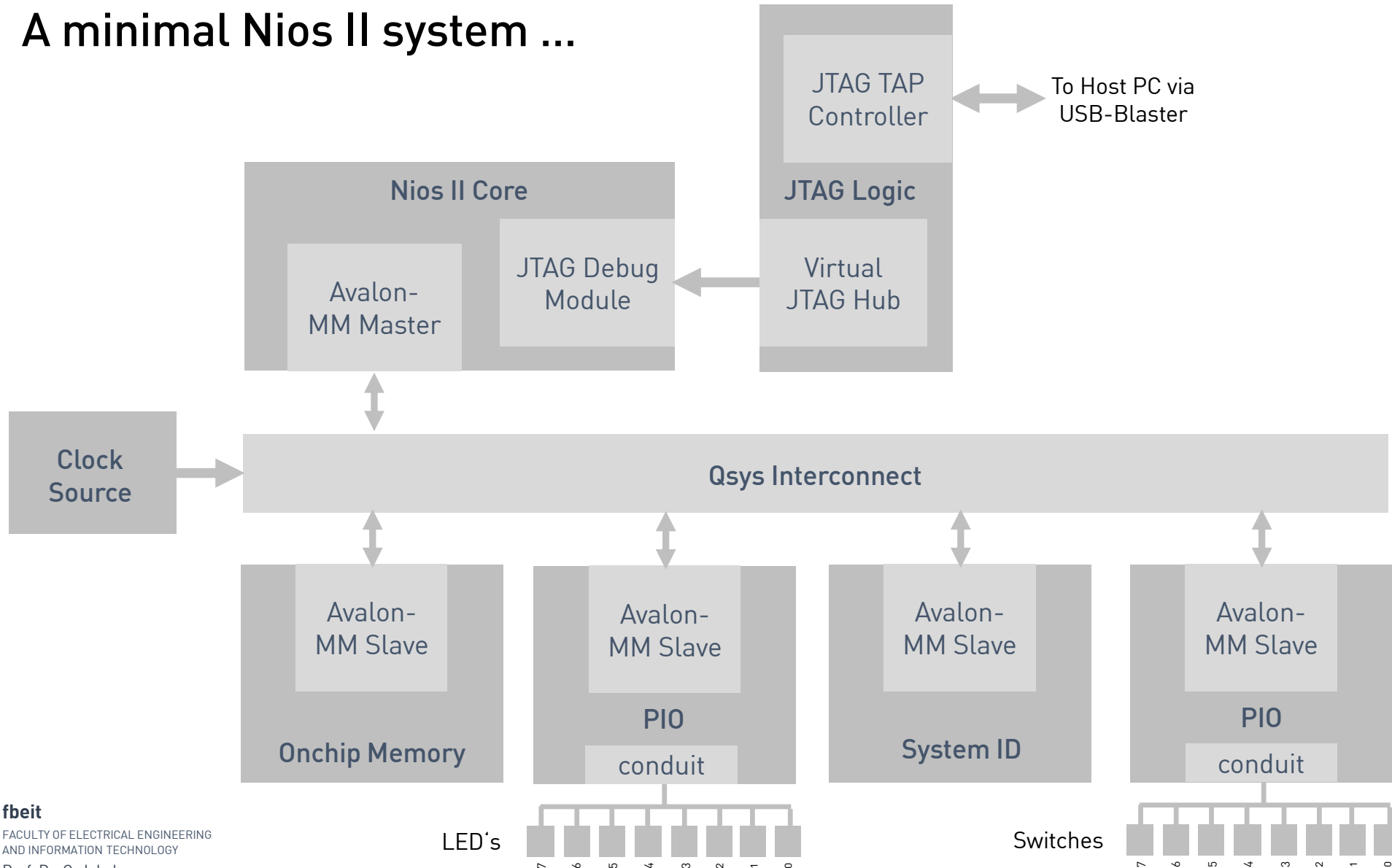
Nios II Memory and I/O Organization



Nios II Performance Optimization

h_da – fbeit - FPGA-based SoC Design

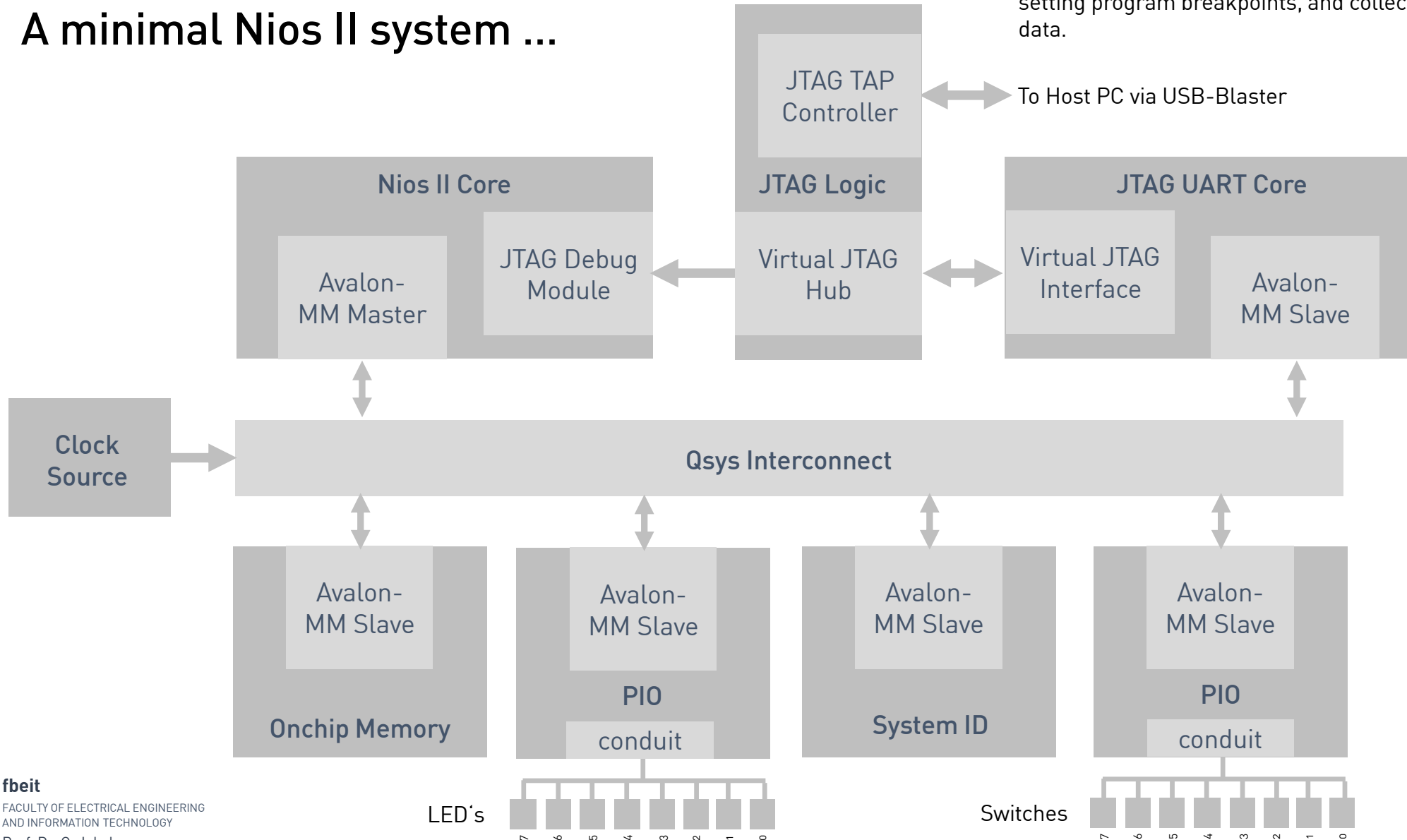
A minimal Nios II system ...



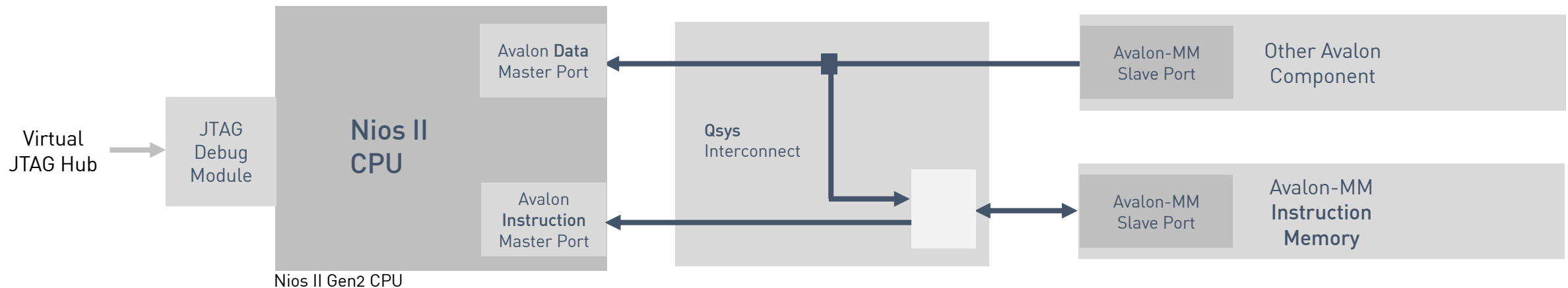
Nios II Performance Optimization

h_da – fbeit - FPGA-based SoC Design

A minimal Nios II system ...

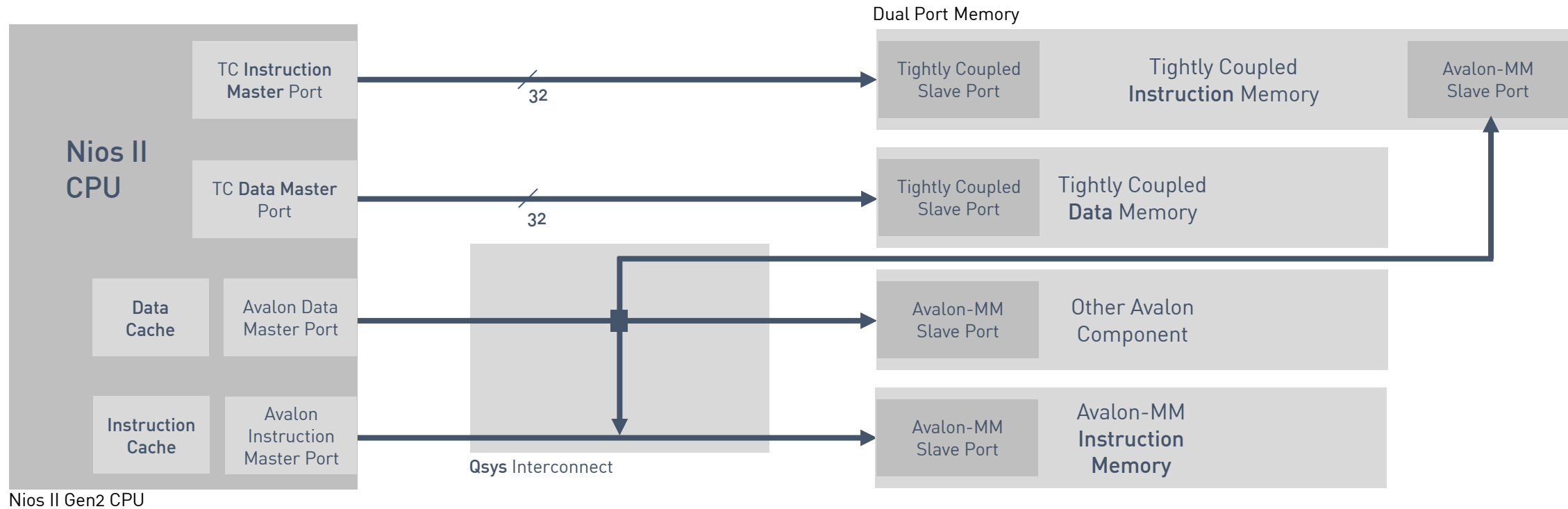


Nios II Gen2 Tightly Coupled Memories



- The Nios II instruction bus is implemented as a 32-bit Avalon-MM master port. **The instruction master port performs a single function: it fetches instructions to be executed by the processor. The instruction master port does not perform any write operations.**
- The instruction master port can issue successive read requests before data has returned from prior requests. The Nios II processor can prefetch sequential instructions and perform branch prediction to keep the instruction pipe as active as possible.
- The instruction master port always retrieves 32 bits of data. The instruction master port relies on dynamic bus-sizing logic contained in the system interconnect fabric. By virtue of dynamic bus sizing, every instruction fetch returns a full instruction word, regardless of the width of the target memory. Consequently, programs do not need to be aware of the widths of memory in the Nios II processor system.

Nios II Gen2 Tightly Coupled Memories



- The **tightly coupled instruction master** is incapable of performing writes because it accesses code for execution only.
- Without a second memory port connected to an Avalon Memory-Mapped (Avalon-MM) data master, the system does not have write access to the tightly coupled instruction memory.
- Without write access, code cannot be downloaded into the tightly coupled memory by the Nios II SBT for Eclipse, which makes development and debugging difficult.

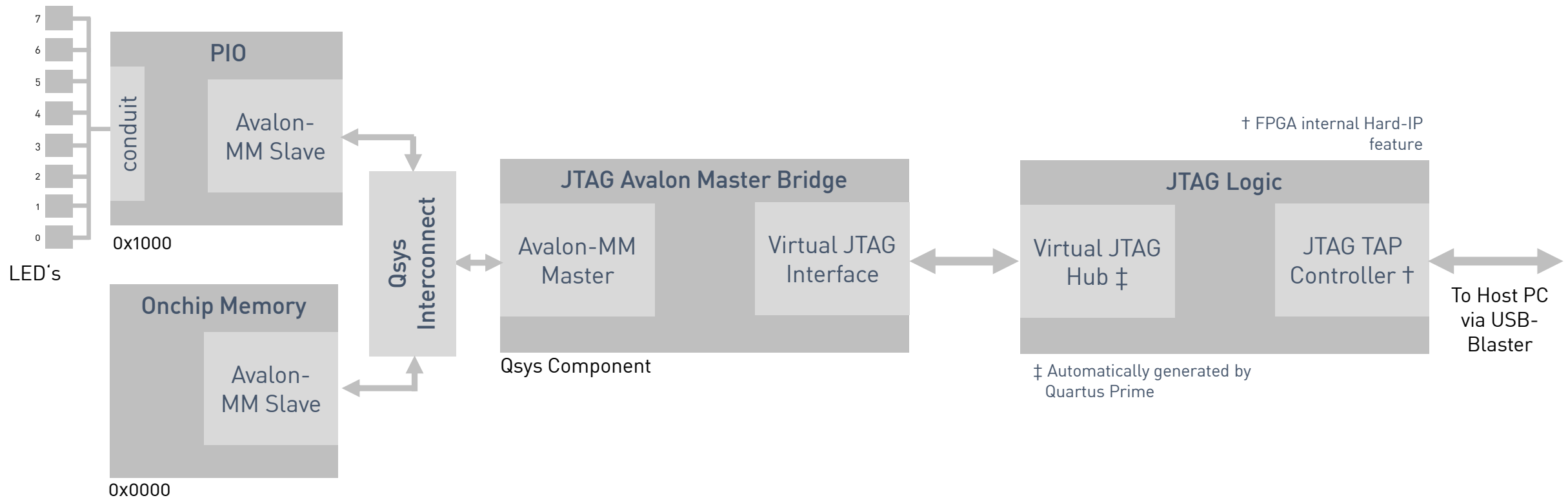
Nios II Performance Optimization

h_da – fbeit - FPGA-based SoC Design

Something in between - Accessing System Components

Nios II Benchmarking - Cyclic Redundancy Check

Fundamental Slave Read Transfers



Nios II Performance Optimization

h_da – fbeit - FPGA-based SoC Design

Nios II – Interrupt Capable Custom Components

Nios II Benchmarking - Cyclic Redundancy Check

Rising Edge Detection

```
1. logic [3:0] sync_reg = 4'b0000;
2. always_ff@(posedge clk)
3.     if(reset_n == 1'b0)
4.         sync_reg <= 4'b0000;
5.     else
6.         sync_reg <= {sync_reg[2:0],d_pulse};
7.
8. assign r_edge = (sync_reg == 4'b0011) ? 1'b1 : 1'b0;
```

1/1

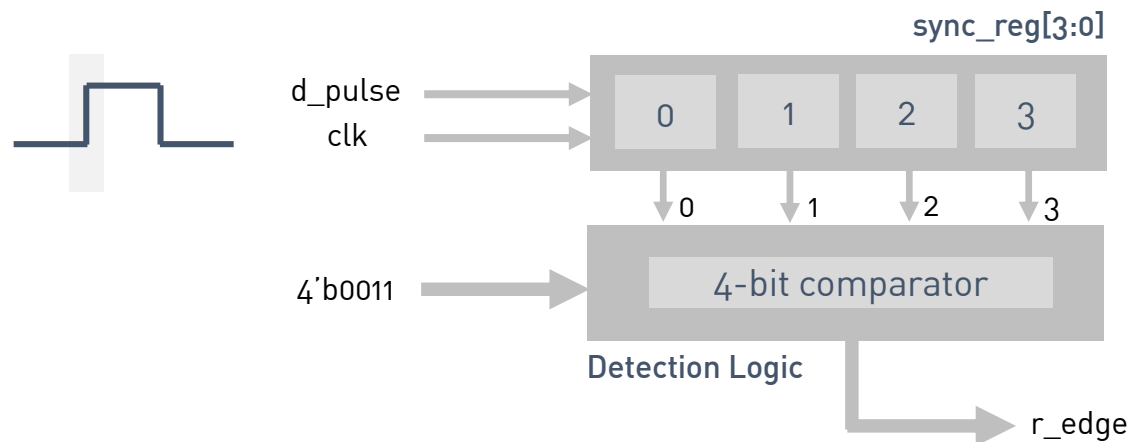
[SystemVerilog] Source Code: rising_edge_detection.v

Rising Edge Detection

```
1. logic [3:0] sync_reg = 4'b0000;
2. always_ff@(posedge clk)
3.     if(reset_n == 1'b0)
4.         sync_reg <= 4'b0000;
5.     else
6.         sync_reg <= {sync_reg[2:0], d_pulse};
7.
8. assign r_edge = (sync_reg == 4'b0011) ? 1'b1 : 1'b0;
```

1/1

[SystemVerilog] Source Code: rising_edge_detection.v



Avalon MM-Slave based Rising Detector with IRQ capability

```
1. module avalon_rising_edge_detection_irq(  
2.     input logic clk, input logic reset_n,  
3.     input logic d_pulse, input logic write,  
4.     input logic [31:0] writedata,  
5.     output logic q_irq_led,  
6.     output logic q_irq  
7. );  
8.  
9. // ### rising edge detection ...  
10. logic [3:0] sync_reg = 4'b0000;  
11. always_ff@(posedge clk)  
12.     if(reset_n == 1'b0)  
13.         sync_reg <= 4'b0000;  
14.     else  
15.         sync_reg <= {sync_reg[2:0], d_pulse};  
16.  
17. logic r_edge;  
18. assign r_edge = (sync_reg == 4'b0011) ? 1'b1 : 1'b0;
```

2/1



[SystemVerilog] Source Code: avalon_rising_edge_detection_irq.v

Avalon MM-Slave based Rising Detector with IRQ capability

```
19. // ### control register ...
20. logic [31:0] ctrl_reg = 0;
21. always_ff@(posedge clk)
22.     if(reset_n == 1'b0)
23.         ctrl_reg <= 0;
24.     else if(write)
25.         ctrl <= writedata[31:0];
26.     else if(r_edge)
27.         ctrl_reg <= {ctrl_reg[31:2],1'b1,ctrl_reg[0]};
28.     else
29.         ctrl_reg <= ctrl_reg;
30.
31. assign q_irq_led = ctrl_reg[1];
32. assign q_irq      = ctrl_reg[1];
33.
34. endmodule
```

2/2



[SystemVerilog] Source Code: `avalon_rising_edge_detection_irq.v`

Nios II Performance Optimization

h_da – fbeit - FPGA-based SoC Design

Avalon MM-Slave based Rising Detector with IRQ capability

```
19. // ### control register ...
20. logic [31:0] ctrl_reg = 0;
21. always_ff@(posedge clk)
22.     if(reset_n == 1'b0)
23.         ctrl_reg <= 0;
24.     else if(write)
25.         ctrl <= writedata[31:0];
26.     else if(r_edge)
27.         ctrl_reg <= {ctrl_reg[31:2], 1'b1, ctrl_reg[0]};
28.     else
29.         ctrl_reg <= ctrl_reg;
30.
31. assign q_irq_led = ctrl_reg[1];
32. assign q_irq      = ctrl_reg[1];
33.
34. endmodule
```

2/2

[SystemVerilog] Source Code: avalon_rising_edge_detection_irq.v

